

Методология экспериментов и аппаратная конфигурация

Для оценки производительности реализованных структур данных были проведены серии benchmark-экспериментов. Основная цель экспериментов заключалась не только в измерении времени работы отдельных операций, но и в сравнении практического поведения разных реализаций при увеличении размера входных данных.

В работе сравнивались собственные реализации структур данных на Python и C++ со встроенными и библиотечными аналогами. Такой подход позволяет рассматривать производительность с нескольких сторон: влияние самого алгоритма, влияние языка программирования и влияние оптимизаций, выполненных в стандартных или сторонних библиотеках.

Для hash table сравнивались следующие реализации:

1. собственная реализация hash table на Python;
2. встроенный словарь Python `dict` ;
3. собственная реализация hash table на C++;
4. библиотечная реализация `boost::unordered_flat_map` .

В собственной реализации hash table использовалось открытое адресование с double hashing. Такой вариант был выбран, поскольку он позволяет уменьшить эффект кластеризации по сравнению с простым linear probing и лучше показывает себя на смешанных сценариях с большим количеством операций вставки, поиска и удаления.

Для binary heap сравнивались следующие реализации:

1. собственная реализация binary heap на Python;
2. библиотечная реализация Python `heapq` ;
3. собственная реализация binary heap на C++;
4. стандартная реализация C++ `std::priority_queue` .

В качестве основной формы кучи рассматривалась min-heap. Для корректного сравнения benchmark-сценарии были подобраны так, чтобы проверять не только простую вставку элементов, но и типичные операции очереди с приоритетом: добавление элементов, извлечение корня, чередование операций и обработку последовательностей разного размера.

Во всех экспериментах использовался единый набор размеров входных данных:

$$N \in \{10\,000, 50\,000, 100\,000, 200\,000, 500\,000, 1\,000\,000, 2\,000\,000, 5\,000\,000, 10\,000\,000, 15\,000\,000\}.$$

Использование нескольких размеров входных данных позволяет наблюдать не только абсолютное время работы, но и характер роста времени при увеличении N . Это важно, потому что различия между реализациями могут быть незаметны на малых размерах, но становиться существенными на больших данных.

Для каждого benchmark-сценария выполнялось по пять повторных запусков:

$$REPEATS = 5.$$

Повторные запуски нужны для уменьшения влияния случайных колебаний времени выполнения. Даже если алгоритм и входные данные одинаковы, итоговое время может немного меняться из-за состояния операционной системы, работы планировщика процессов, кэша процессора и других низкоуровневых факторов.

Время измерялось только для самой benchmark-операции или benchmark-сценария. Запись результатов в файл, последующая обработка CSV, построение таблиц и построение графиков не входили в измеряемый участок. Это позволяет сравнивать именно время выполнения структур данных, а не накладные расходы вспомогательных этапов.

Результаты benchmark-экспериментов сохранялись в формате CSV. Общий pipeline обработки результатов имел следующий вид:

raw CSV $\rightarrow$ processed CSV $\rightarrow$ tables $\rightarrow$ plots.

Сначала сохранялись сырые результаты отдельных запусков. Затем для каждой комбинации структуры данных, реализации, размера входных данных и benchmark-сценария вычислялись агрегированные статистики. После этого строились таблицы и графики, используемые для анализа.

Для обработки результатов использовались следующие статистические характеристики:

1. `mean` --- среднее время выполнения;
2. `median` --- медианное время выполнения;
3. `std` --- стандартное отклонение;
4. `sem` --- стандартная ошибка среднего.

Среднее значение использовалось в таблицах, так как оно показывает общее усредненное время выполнения по нескольким запускам. Медиана использовалась как основная точка на графиках, поскольку она менее чувствительна к отдельным выбросам. Стандартное отклонение и стандартная ошибка среднего использовались для оценки устойчивости измерений.

На графиках по оси X откладывался размер входных данных N , а по оси Y --- время выполнения в секундах. Для каждой реализации строилась отдельная линия. Также на графиках отображались error bars, построенные на основе стандартной ошибки среднего. В большинстве случаев эти интервалы почти не видны, что связано с малым разбросом между повторными запусками. Это означает, что измерения были достаточно стабильными.

Дополнительно на графиках использовался inset zoom --- увеличенный фрагмент области малых значений N . Это необходимо, потому что при больших размерах входных данных различия во времени становятся настолько значительными, что поведение реализаций на малых размерах может визуально сжиматься и становиться плохо различимым на основном графике.

Все графики строились в линейной шкале. Такой формат был выбран для наглядного сравнения абсолютного времени выполнения разных реализаций. Линейная шкала позволяет явно увидеть, насколько одна реализа-

ция быстрее или медленнее другой в секундах, что особенно удобно при практическом сравнении производительности.

Эксперименты проводились на одном и том же устройстве. Во время выполнения benchmark-программ не взаимодействовал с ноутбуком и не запускал дополнительные ресурсоемкие задачи. Это было сделано для уменьшения влияния внешних факторов на результаты измерений.

Аппаратная конфигурация устройства:

1. устройство: MacBook Pro 13-inch, 2018;
2. процессор: Intel Core i5, 4 ядра, 2.3 GHz;
3. оперативная память: 16 GB LPDDR3, 2133 MHz;
4. графика: Intel Iris Plus Graphics 655;
5. архитектура: x86_64.

Программная конфигурация:

1. операционная система: macOS;
2. версия Python: Python 3.11.4 ;
3. компилятор C++: Apple clang 17.0.0 ;
4. стандарт C++: C++17 ;
5. среда разработки для Python: PyCharm 2023.2.3 Community Edition;
6. среда разработки для C++: CLion 2025.2.

C++-проект собирался с помощью CMake. В конфигурации проекта был указан стандарт C++17 . Явные флаги оптимизации компилятора, такие как -O2 или -O3 , отдельно не задавались. Поэтому при интерпретации результатов важно учитывать, что измерения отражают производительность в использованной конфигурации сборки.

Использование единой аппаратной платформы, одинакового набора размеров входных данных и одинакового числа повторных запусков позволяет сделать сравнение реализаций более корректным. При этом результаты не следует воспринимать как универсальные для всех компьютеров и всех компиляторов: на другой аппаратной конфигурации абсолютные значения времени могут отличаться. Однако относительные различия между реализациями позволяют выявить основные причины расхождения между теоретической сложностью и практической производительностью.

Таким образом, методология экспериментов была построена так, чтобы сравнивать реализации не по единичному запуску, а по серии измерений с последующей статистической обработкой. Это позволяет сделать выводы более устойчивыми и связать наблюдаемое поведение структур данных с особенностями языка программирования, реализации, организации памяти и библиотечных оптимизаций.